MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE RUSSIAN FEDERATION

Federal State Autonomous Educational Institution of Higher Education
“Peter the Great St. Petersburg Polytechnic University”

Institute of Computer Science and Cybersecurity

Higher School of Technology and Artificial Intelligence

Programme: 02.03.01 Mathematics and Computer Science

Course Work Report
on the subject “Graph Theory”
Implementation of an Expert System:
“Choosing a Pet Based on Given Characteristics”

Student,
group 5130201/40001 _____ **Ovi Md Shamin Yasir**

Supervisor,
_____ **Vostrov Aleksey Vladimirovich**

«____»_____ 2026

Saint Petersburg, 2026

Contents

Introduction

Expert systems are knowledge-based systems that operate within specific subject domains.

An expert system encodes the knowledge of a highly qualified specialist in a particular domain and is used for consultation, decision support, and solving ill-structured problems.

The goal of this course work is to implement an expert system in the CLIPS environment based on the production model for the domain “Choosing a pet based on given characteristics”.

Minimum binary decision tree size: 4 levels, 30 nodes.

1 Subject Domain

The chosen subject domain is "choosing a pet based on given characteristics". This knowledge domain contains information used to match the most suitable pet to a user depending on their lifestyle, living conditions, and personal preferences.

The classification attributes are chosen so that the user can unambiguously identify the most suitable pet. Key classification criteria: importance of daily interaction, willingness to walk and train the animal, preference for active or passive pet, tolerance for nocturnal activity, interest in an aquarium or a bird.

Why This Topic Was Chosen

Choosing a pet is a typical ill-structured problem: there is no single algorithm for solving it, and the result depends on many subjective factors — lifestyle, living conditions, available time, and personal preferences. Such problems are ideally suited to the production model of an expert system, since each characteristic can be expressed unambiguously as an IF-THEN rule:

IF (daily interaction is important) AND (time available for walks)
AND (large animal is acceptable)
THEN recommend a Dog

The topic is relevant: in Bangladesh the number of pet owners has more than doubled in the last three years. The pet products market amounts to approximately 2 billion taka per year, growing at over 20% annually [1].

How the Animals Were Selected

All 16 animals included in the system were selected based on their popularity and availability in Bangladesh, confirmed by reliable sources:

- According to The Daily Star (Bangladesh) [1], approximately 90% of pet owners keep cats — the most popular pet in the country. The pet market is growing at over 20% per year.
- A 2024 academic study (ResearchGate) [2] found: cats — 56.2% of owners, birds — 35.5%, dogs — 11.1%, rabbits — 5% of all pets.
- According to Bikroy.com [4] (the largest online marketplace in Bangladesh), the following animals are actively sold: dogs, cats, rabbits, parrots, pigeons, myna birds, canaries, aquarium fish, turtles, hamsters, guinea pigs, fancy rats, cockatiels, budgerigars, lovebirds, and goldfish.
- According to BdFISH [5], Bangladesh has more than 70 species of ornamental and aquarium fish actively sold in pet shops in Dhaka and Rajshahi.
- Katabon Online (Dhaka) [6] confirms the availability of all 16 species in physical pet stores.

Selection criterion: widely available, accessible, and safe domestic animals in Bangladesh, suitable for keeping in an apartment or house.

List of Animals (Expert System Facts)

The following is the complete list of possible outcomes of the expert system:

1. Dog

2. Rabbit
3. Cat
4. Guinea Pig
5. Hamster
6. Fancy Rat
7. Pigeon
8. Myna Bird
9. Aquarium Fish
10. Goldfish
11. Turtle
12. Cockatiel
13. Parrot
14. Canary
15. Lovebird
16. Budgerigar

2 Mathematical Description

2.1 Expert System and Its Properties

An **expert system** is a computer system that uses the knowledge of domain experts to solve complex problems and provide recommendations or predictions.

Expert systems are primarily based on heuristic search rather than the execution of a known algorithm. This is one of the main advantages of expert system technology over the traditional approach to program development, and it is what allows them to handle the tasks they are designed for.

An expert system operates in two main modes:

- knowledge acquisition mode;
- problem-solving mode (also called consultation mode, or usage mode).

In knowledge acquisition mode, the expert system is operated by a domain expert assisted by a knowledge engineer. In this mode, the expert uses the knowledge acquisition component to populate the system with knowledge (data), which then allows the system to solve problems from that domain independently in problem-solving mode.

In problem-solving mode (also called consultation mode), the end user interacts directly with the expert system, interested in the final result and sometimes in how it was obtained. Depending on the purpose of the system, the user does not necessarily need to be a specialist in the problem domain — they consult the expert system for a result without possessing sufficient expert knowledge.

Components of the expert system structure:

- **User Interface**: provides interaction between the user and the expert system. It may be graphical, text-based, or voice-based, allowing the user to ask questions, receive answers, enter data, and interact with the system. The interface must be intuitive and user-friendly.
- **Knowledge Base**: knowledge consists of rules, laws, and patterns obtained through professional activity in a subject domain. The knowledge base is a database containing inference rules and information about human experience and expertise. In other words, it is a set of patterns that establish links between input and output information.
- **Data**: data is a collection of facts and ideas in a formalised form. Data forms the basis for patterns used in prediction and forecasting. Advanced intelligent systems can learn from this data, adding new knowledge to the knowledge base.
- **Logical Inference Mechanism (Inference Subsystem)**: performs analysis and derives new knowledge by matching source data from the database against rules from the knowledge base. The logical inference mechanism can be represented conceptually as $\langle A, B, C, D \rangle$:
  - A — a function that selects patterns from the knowledge base and facts from the database;
  - B — a rule-checking function that determines the set of facts to which the rules are applicable;
  - C — a function that determines the order of rule application when multiple rules match the same facts;
  - D — a function that executes the action.

- **Inference Engine**: the software component that identifies rule antecedents (where applicable) that match current facts. To do this, the inference engine:
  - selects rules whose conditions match current facts;
  - ranks the selected rules by priority;
  - fires the rule with the highest priority.
- **Intelligent Editor**: supports the knowledge base update mode. Allows the domain expert, assisted by a knowledge engineer, to add new rules and facts to the system.

2.2 Production Model of Knowledge Representation

Production knowledge models are closely related to logical models, which makes it possible to organise highly effective logical inference procedures.

The traditional production knowledge model includes the following basic components:

1. a set of rules (or productions) representing the knowledge base of the production system;
2. working memory, which stores the initial facts as well as facts derived from them by the inference mechanism;
3. the inference mechanism itself, which derives new facts from existing facts according to the available inference rules.

- The core of the production knowledge model is the production (rule):
  IF $\langle$*condition*$\rangle$, THEN $\langle$*action 1*$\rangle$, ELSE $\langle$*action 2*$\rangle$
- A production consists of two parts: the condition — the antecedent, and the action — the consequent.
- Conditions can be combined using the logical operators AND and OR.
- Antecedents and consequents are formed from attributes and values.
- The database of the production system stores rules whose truth has been established in advance for a given problem. A rule fires when the facts in the database match the rule's antecedent. The result of firing is stored back in the database.
- General form of the production model: $N = \langle A, U, C, I, R\rangle$, where N is the production name, A is the application domain, U is the applicability condition, C is the production core (the IF-THEN rule), I is the post-conditions, R is a comment.

2.3 Binary Decision Tree

The expert system is represented as a binary decision tree — an acyclic directed graph with cause-and-effect relationships. In a decision tree, questions are decision nodes and final outcomes are leaves.

The operation of the expert system corresponds to a path through the graph. The path consists of a sequence of steps at each of which the user decides which arc to follow from the current node. All questions are binary (yes/no).

Description of the decision tree:

- **Nodes**: nodes represent states or decisions in the decision tree. Each node has certain properties, which may include conditions, variables, or factors on which the decision is based.

- **Edges**: edges represent transitions or branches between nodes. They define the logical sequence of decision-making and the links between different states. **There are two types of edges: a dashed edge represents the answer "no" and a solid edge represents the answer "yes"**.
- **Conditions**: conditions define the logical expressions or predicates that must be true in order to follow a given edge. They may be based on variable values or factors defined in the expert system.
- **Rules**: rules are part of nodes and define the decision logic in specific states. They can be expressed as condition-action pairs or logical rules that specify what actions or conclusions to draw based on given conditions.
- **Conclusions**: conclusions are the outcomes or decisions that can be reached at a terminal node or leaf of the decision tree. They can be specific values, classifications, or actions to be taken based on the decisions made.

Numerical characteristics of the tree: number of nodes — 31, number of levels — 4, number of facts (leaves) — 16, number of rules — 15.

The binary decision tree constructed during this course work is shown as a diagram in Fig. 1.

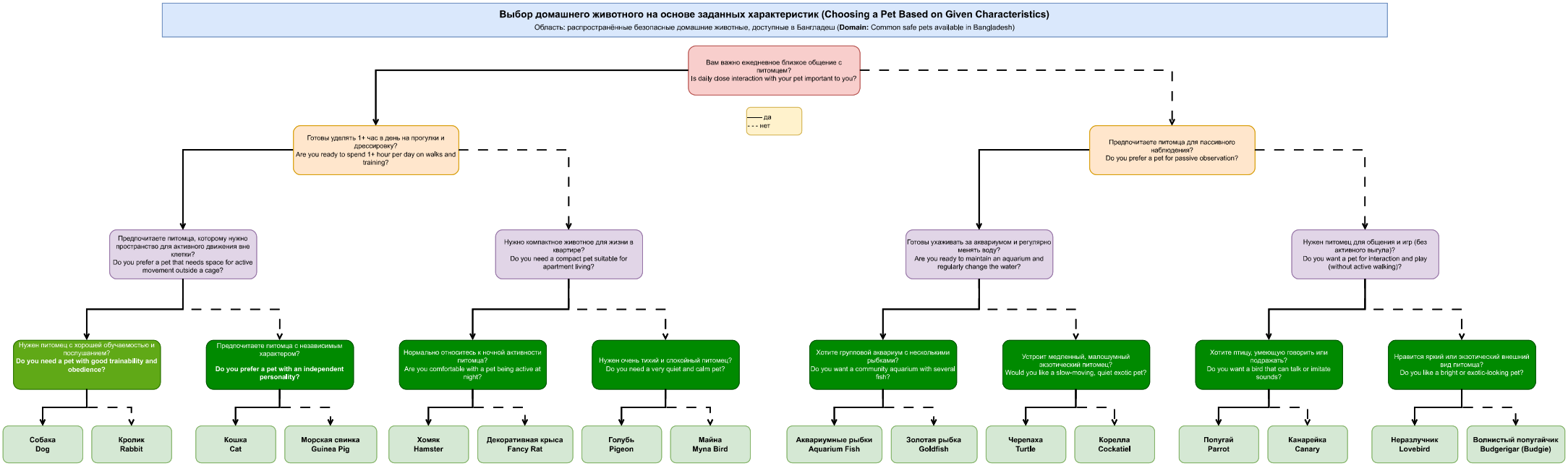

Figure 1 — Binary decision tree

3 Implementation Details

The expert system is implemented in the CLIPS environment (C Language Integrated Production System) based on the production model. The knowledge base consists of 31 rules (`defrule`): 15 rules implement the decision tree nodes and 16 rules implement the leaf nodes with recommendations. CLIPS working memory stores the current facts about the user's preferences. The inference engine automatically selects and fires matching rules. This section describes the key functions and rules of the program [10, 8, 9].

3.1 General Input Function with Validation

The `ask-question` function is a universal input function that validates user input. It accepts the question text `?question` and a list of allowed values `$?allowed-values`. The function prints the question, reads the answer using `read`, and converts it to lowercase using `lowcase`. If the answer is not in the list of allowed values (`member$`), an error message is displayed and the question is repeated in a `while` loop. Returns the accepted answer. The code is shown in Listing 1.

```
(deffunction ask-question (?question $?allowed-values)
    (printout t ?question)
    (bind ?answer (read))
    (if (lexemep ?answer)
        then (bind ?answer (lowcase ?answer)))
    (while (not (member$ ?answer ?allowed-values)) do
        (printout t "  Invalid input. Please enter: " ?allowed-values crlf)
        (printout t ?question)
        (bind ?answer (read))
        (if (lexemep ?answer)
            then (bind ?answer (lowcase ?answer))))
    ?answer)
```

Listing 1. The ask-question function

3.2 Yes/No Wrapper Function

The `yesno` function is a wrapper around `ask-question`. It accepts only the question text and calls `ask-question` with a fixed list of allowed values (`yes no y n`). It normalises the answer: if the user entered `y` or `yes`, the symbol `yes` is returned; otherwise `no` is returned. This ensures consistency: all subsequent rules in the system work only with the symbols `yes` and `no`, regardless of what the user actually typed. The code is shown in Listing 2.

```
(deffunction yesno (?question)
    (bind ?response (ask-question ?question yes no y n))
    (if (or (eq ?response yes) (eq ?response y))
        then yes
        else no))
```

Listing 2. The yesno function

3.3 Recommendation Print Rule

The `print-rec` rule has the highest priority (`salience 1000`), which guarantees that it fires before any other rule as soon as a `(rec ...)` fact appears in working memory. The rule prints the final recommendation, retracts the `(rec ...)` fact from working memory, and asserts the `(restart)` fact, thereby triggering the restart rule. The code is shown in Listing 3.

```
(defrule print-rec
    (declare (salience 1000))
    ?rec <- (rec ?item)
    =>
    (printout t crlf "Recommended pet: " ?item crlf crlf)
    (retract ?rec)
    (assert (restart)))
```

Listing 3. The print-rec rule

3.4 Restart Rule

The `restart-system` rule fires when the `(restart)` fact appears in working memory. It retracts the `(restart)` fact and asks the user whether they want a new recommendation, using `ask-question` with input validation. If the answer is positive (`yes` or `y`), the program title is printed and a new fact containing the answer to the first question is asserted in working memory, starting a new session. If the answer is negative, the message `Goodbye!` is displayed and the inference engine is stopped with `halt`. The code is shown in Listing 4.

```
(defrule restart-system
    ?r <- (restart)
    =>
    (retract ?r)
    (bind ?a (ask-question "Would you like a new recommendation? (yes/no): "
                           yes no y n))
    (if (or (eq ?a yes) (eq ?a y))
        then
            (printout t crlf
                "=== Expert System: Pet Selection (Bangladesh) ===" crlf crlf)
            (assert (interaction
                (yesno "Is daily close interaction with your pet important to you? (yes/no):
 ")))
        else
            (printout t crlf "Goodbye!" crlf)
            (halt)))
```

Listing 4. The restart-system rule

3.5 Decision Tree Rules

Each tree node is implemented as a pair of rules — one for the "yes" answer and one for the "no" answer. Each rule: binds the matching fact to the variable `?f`, retracts it from working memory (`retract`), asks the next question via `yesno`, and asserts a new fact (`assert`). Retracting the fact immediately after the rule fires ensures that working memory is clean at the end of each session, which guarantees correct behaviour on restart. The entry rule `first` fires only when working memory is completely empty and prints the program title. Listing 5 shows a fragment demonstrating the entry rule and the first two levels of the tree; the full implementation is given in Appendix A1.

```
; Entry point -- fires only when working memory is empty
(defrule first ""
    (not (interaction ?))
    (not (rec ?))
    (not (restart))
    =>
    (printout t "=== Expert System: Pet Selection (Bangladesh) ===" crlf crlf)
    (assert (interaction
```

```
        (yesno "Is daily close interaction with your pet important to you? (yes/no): "))))

; Level 2 -- n2 (yes branch)
(defrule interaction-yes ""
    ?f <- (interaction yes) (not (rec ?)) =>
    (retract ?f)
    (assert (walks
        (yesno "Are you ready to spend 1+ hour per day on walks and training? (yes/no): ")))
    )

; Level 2 -- n3 (no branch)
(defrule interaction-no ""
    ?f <- (interaction no) (not (rec ?)) =>
    (retract ?f)
    (assert (passive
        (yesno "Do you prefer a pet for passive observation? (yes/no): "))))
```

Listing 5. Fragment of decision tree rules (entry rule and first two levels)

3.6 Leaf Rules (Recommendations)

The 16 leaf rules implement the leaf nodes of the decision tree. Each rule checks for the presence of a level-4 fact and the absence of a `(rec ...)` fact. When fired, it retracts its own fact and asserts `(rec "Animal Name")`. This immediately activates the `print-rec` rule with priority 1000. The code is shown in Listing 6.

```
(defrule trainable-yes  "" ?f <- (trainable yes)  (not (rec ?)) =>
    (retract ?f) (assert (rec "Dog")))
(defrule trainable-no   "" ?f <- (trainable no)   (not (rec ?)) =>
    (retract ?f) (assert (rec "Rabbit")))

(defrule independent-yes "" ?f <- (independent yes) (not (rec ?)) =>
    (retract ?f) (assert (rec "Cat")))
(defrule independent-no  "" ?f <- (independent no)  (not (rec ?)) =>
    (retract ?f) (assert (rec "Guinea Pig")))

; ... (remaining 12 leaf rules follow the same pattern)

(defrule bright-yes "" ?f <- (bright yes) (not (rec ?)) =>
    (retract ?f) (assert (rec "Lovebird")))
(defrule bright-no  "" ?f <- (bright no)  (not (rec ?)) =>
    (retract ?f) (assert (rec "Budgerigar")))
```

Listing 6. Leaf rules (fragment)

4 Program Results

The result of the work is a complete program implemented in the CLIPS environment. It is launched using the commands `(load "pet_expert.clp")`, `(reset)`, `(run)` in the CLIPS interpreter. After launch the title is displayed and the first question (the root of the binary decision tree) is asked. The user answers `yes` (or `y`) for "yes" and `no` (or `n`) for "no". The program validates input: if an invalid value is entered, an error message is shown and the question is repeated. After passing through four levels of the tree, the system displays a recommendation and offers to start a new session. Examples of program execution are shown below.

4.1 Example 1: path yes-yes-yes-yes, recommendation — Dog

Fig. 2 shows the first program session. The user answers `y` to all four questions, forming the path yes → yes → yes → yes through the decision tree: node n1 (interaction) → node n2 (walks) → node n4 (space) → node n8 (trainable). The leaf rule `trainable-yes` fires, asserts the fact `(rec "Dog")`, and the `print-rec` rule with priority `salience 1000` immediately fires, printing the recommendation *Dog*. After the recommendation the `restart-system` rule fires, asking whether the user wishes to start a new session.

```
CLIPS> (reset)
CLIPS> (run)
=== Expert System: Pet Selection (Bangladesh) ===

Is daily close interaction with your pet important to you? (yes/no): y
Are you ready to spend 1+ hour per day on walks and training? (yes/no): y
Do you prefer a pet that needs space for active movement outside a cage? (yes/no): y
Do you need a pet with good trainability and obedience? (yes/no): y

Recommended pet: Dog

Would you like a new recommendation? (yes/no):
```

Figure 2. Example 1: path yes-yes-yes-yes, recommendation — Dog

4.2 Example 2: second session and program exit

Fig. 3 shows a second session with different answers followed by a program exit. After the user answers `y` at the restart prompt, the `restart-system` rule asserts a new `(interaction ...)` fact, fully restarting the dialogue from the root of the tree. Because every rule retracts its own triggering fact using the pattern `?f <- (fact) => (retract ?f)`, working memory is completely clean at the start of each new session — no facts from the previous run remain. The user then provides different answers and receives a new recommendation. When the user finally answers `n` at the restart prompt, the rule calls `(halt)`, stopping the CLIPS inference engine and ending the program with the message *Goodbye!*

```
Would you like a new recommendation? (yes/no): y

=== Expert System: Pet Selection (Bangladesh) ===

Is daily close interaction with your pet important to you? (yes/no): n
Do you prefer a pet for passive observation? (yes/no): n
Do you want a pet for interaction and play without active walking? (yes/no): y
Do you want a bird that can talk or imitate sounds? (yes/no): n

Recommended pet: Canary

Would you like a new recommendation? (yes/no): n

Goodbye!
CLIPS>
```

Figure 3. Example 2: second session with new answers and program exit

4.3 Example 3: invalid input handling

Fig. 4 demonstrates the built-in input validation mechanism. The user enters the arbitrary string `asdf` instead of one of the accepted values. The `ask-question` function checks the answer using `(member$ ?answer $?allowed-values)`: since `asdf` is not in the list `(yes no y n)`, the `while` loop condition remains true. The function displays the message *"Invalid input. Please enter: (yes no y n)"* and repeats the question. The dialogue continues only after the user provides a valid answer. This mechanism is implemented entirely in the `ask-question` function and applies uniformly to all questions in the session, including the restart prompt.

```
=== Expert System: Pet Selection (Bangladesh) ===

Is daily close interaction with your pet important to you? (yes/no): n
Do you prefer a pet for passive observation? (yes/no): y
Are you ready to maintain an aquarium and regularly change the water? (yes/no): asdf
  Invalid input. Please enter: (yes no y n)
Are you ready to maintain an aquarium and regularly change the water? (yes/no): 3
  Invalid input. Please enter: (yes no y n)
Are you ready to maintain an aquarium and regularly change the water? (yes/no): 5
  Invalid input. Please enter: (yes no y n)
Are you ready to maintain an aquarium and regularly change the water? (yes/no):

y
Do you want a community aquarium with several fish? (yes/no): y

Recommended pet: Aquarium Fish

Would you like a new recommendation? (yes/no):
```

Figure 4. Example 3: invalid input handling — the system shows an error message and repeats the question

Conclusion

As a result of this course work, an expert system was implemented in the CLIPS environment on the topic "Choosing a pet based on given characteristics (Bangladesh)" [1, 2, 3]. The system is represented as a binary decision tree with the following numerical characteristics: number of nodes — 31, number of levels — 4, number of facts — 16, number of rules — 15. The knowledge of the implemented system is represented using the production model: the knowledge base consists of 31 rules, working memory stores the current session facts, and the CLIPS inference engine automatically controls rule activation. The system allows the user to unambiguously identify the most suitable pet based on their lifestyle, living conditions, and personal preferences.

Advantages

Advantages of using CLIPS and the production model:

- Knowledge (rules) is separated from the control mechanism (the CLIPS inference engine): adding a new node to the tree requires only adding a new pair of rules, without modifying the control logic.
- The binary decision tree traversal algorithm is efficient: exactly 4 questions are required to obtain a recommendation, corresponding to the number of tree levels.
- The priority mechanism (`salience`) ensures a predictable rule firing order: the recommendation print rule always fires first.

Disadvantages

- Adding or removing a node requires redesigning the decision tree and adding the corresponding rules, so the system is not completely flexible.
- The expert system is suitable for problems with a limited number of factors and simple interactions; more complex problems require more advanced knowledge representation methods — semantic networks and frame-based models [9].

Scalability

The program can be extended by adding new questions and animals, refining the classification criteria, or expanding the binary decision tree. Another approach to scaling is to switch to more complex knowledge representation structures in CLIPS: using `deftemplate` for structured facts, or connecting an external knowledge base [8].

References

[1] "Market for pet food, accessories growing" / The Daily Star. — URL: https://www.thedailystar.net/business/economy/news/market-pet-food-accessories-growing-3422306 (accessed: 11.05.2026)

[2] The evolving paradigm of pet animal rearing in Bangladesh / ResearchGate. — URL: https://www.researchgate.net/publication/388758458 (accessed: 11.05.2026)

[3] Navigating Pet Welfare in Bangladesh: A Multilayered Analysis of Ownership, Ethics and Service Accessibility / ResearchGate. — URL: https://www.researchgate.net/publication/398122556 (accessed: 11.05.2026)

[4] Animals & Pets for sale in Bangladesh / Bikroy.com. — URL: https://bikroy.com/en/ads/bangladesh/pets-animals (accessed: 11.05.2026)

[5] Aquarium Fishes of Bangladesh: Part-1 / BdFISH Feature. — URL: https://en.bdfish.org/2011/01/aquarium-fish-bangladesh-1/ (accessed: 11.05.2026)

[6] Katabon Online — Best Quality Pet Supplies (Dhaka pet market). — URL: https://katabononline.com/ (accessed: 11.05.2026)

[7] The Rising Trend of Pet Ownership in Bangladesh / Daily Business Eye. — URL: https://www.dailybusinesseye.com/todayspaper/paper/4183 (accessed: 11.05.2026)

[8] Khabarov S. P. CLIPS — a language for building expert systems. — Saint Petersburg, 2013. — 80 p.

[9] Gavrilova T. A., Chasikov A. P. Development of Expert Systems. The CLIPS Environment. — Saint Petersburg: BHV-Petersburg, 2003. — 393 p.

[10] Section "Telematika" / electronic text. — URL: https://tema.spbstu.ru/tgraph/ (accessed: 11.05.2026)

Appendix A1. File pet_expert.clp

```
; ============================================================
; Expert System: Choosing a Pet (Bangladesh)
; Binary decision tree  -- 4 levels, 15 decision nodes, 16 recommendations
; ------------------------------------------------------------
; Usage:  CLIPS> (load "pet_expert.clp")
;         CLIPS> (reset)
;         CLIPS> (run)
; Input:  yes / y  for YES
;         no  / n  for NO
; ============================================================


; ------------------------------------------------------------
; 3.1  General question function
;      Asks ?question, reads answer, loops until it matches
;      one of $?allowed-values (case-insensitive).
; ------------------------------------------------------------
(deffunction ask-question (?question $?allowed-values)
    (printout t ?question)
    (bind ?answer (read))
    (if (lexemep ?answer)
        then (bind ?answer (lowcase ?answer)))
    (while (not (member$ ?answer ?allowed-values)) do
        (printout t "  Invalid input. Please enter: " ?allowed-values crlf)
        (printout t ?question)
        (bind ?answer (read))
        (if (lexemep ?answer)
            then (bind ?answer (lowcase ?answer))))
    ?answer)


; ------------------------------------------------------------
; 3.2  Yes/No wrapper
;      Calls ask-question with allowed values yes/no/y/n.
;      Returns the symbol  yes  or  no.
; ------------------------------------------------------------
(deffunction yesno (?question)
    (bind ?response (ask-question ?question yes no y n))
    (if (or (eq ?response yes) (eq ?response y))
        then yes
        else no))


; ------------------------------------------------------------
; 3.3  Restart rule
;      Fires when (restart) fact appears.
;      Asks the user whether to run again.
;      If yes  -- starts a new session by asserting n1 answer.
;      If no   -- prints goodbye and halts.
; ------------------------------------------------------------
(defrule restart-system
    ?r <- (restart)
    =>
    (retract ?r)
    (bind ?a (ask-question "Would you like a new recommendation? (yes/no): " yes no y n))
    (if (or (eq ?a yes) (eq ?a y))
        then
            (printout t crlf "=== Expert System: Pet Selection (Bangladesh) ===" crlf crlf)
```

```
            (assert (interaction
                (yesno "Is daily close interaction with your pet important to you? (yes/no):
     ")))
        else
            (printout t crlf "Goodbye!" crlf)
            (halt)))


; ------------------------------------------------------------
; 3.4  Print recommendation
;      Highest salience so it fires before any other rule.
;      Prints the result, retracts it, and triggers restart.
; ------------------------------------------------------------
(defrule print-rec
    (declare (salience 1000))
    ?rec <- (rec ?item)
    =>
    (printout t crlf "Recommended pet: " ?item crlf crlf)
    (retract ?rec)
    (assert (restart)))


; ============================================================
; 3.5  Question rules  (one rule per decision node)
;      Each rule binds its matching fact with ?f and retracts
;      it immediately  -- so working memory is fully clean after
;      every session and restart works without leftover facts.
;      Solid edge  = YES answer
;      Dashed edge = NO  answer
; ============================================================

; --- Entry point  -- Level 1  -- n1 ----------------------------
(defrule first ""
    (not (interaction ?))
    (not (rec ?))
    (not (restart))
    =>
    (printout t "=== Expert System: Pet Selection (Bangladesh) ===" crlf crlf)
    (assert (interaction
        (yesno "Is daily close interaction with your pet important to you? (yes/no): "))))

; --- Level 2  -- n2 / n3 ---------------------------------------
(defrule interaction-yes ""
    ?f <- (interaction yes) (not (rec ?)) =>
    (retract ?f)
    (assert (walks
        (yesno "Are you ready to spend 1+ hour per day on walks and training? (yes/no): ")))
    )

(defrule interaction-no ""
    ?f <- (interaction no) (not (rec ?)) =>
    (retract ?f)
    (assert (passive
        (yesno "Do you prefer a pet for passive observation? (yes/no): "))))

; --- Level 3  -- n4 / n5 / n6 / n7 -----------------------------
(defrule walks-yes ""
    ?f <- (walks yes) (not (rec ?)) =>
    (retract ?f)
    (assert (space
```

```
        (yesno "Do you prefer a pet that needs space for active movement outside a cage? (yes/no): "))))

(defrule walks-no ""
    ?f <- (walks no) (not (rec ?)) =>
    (retract ?f)
    (assert (compact
        (yesno "Do you need a compact pet suitable for apartment living? (yes/no): "))))

(defrule passive-yes ""
    ?f <- (passive yes) (not (rec ?)) =>
    (retract ?f)
    (assert (aquarium
        (yesno "Are you ready to maintain an aquarium and regularly change the water? (yes/no): "))))

(defrule passive-no ""
    ?f <- (passive no) (not (rec ?)) =>
    (retract ?f)
    (assert (playpet
        (yesno "Do you want a pet for interaction and play without active walking? (yes/no): "))))

; --- Level 4  -- n8 / n9 / n10 / n11 / n12 / n13 / n14 / n15
(defrule space-yes ""
    ?f <- (space yes) (not (rec ?)) =>
    (retract ?f)
    (assert (trainable
        (yesno "Do you need a pet with good trainability and obedience? (yes/no): "))))

(defrule space-no ""
    ?f <- (space no) (not (rec ?)) =>
    (retract ?f)
    (assert (independent
        (yesno "Do you prefer a pet with an independent personality? (yes/no): "))))

(defrule compact-yes ""
    ?f <- (compact yes) (not (rec ?)) =>
    (retract ?f)
    (assert (nocturnal
        (yesno "Are you comfortable with a pet being active at night? (yes/no): "))))

(defrule compact-no ""
    ?f <- (compact no) (not (rec ?)) =>
    (retract ?f)
    (assert (quiet
        (yesno "Do you need a very quiet and calm pet? (yes/no): "))))

(defrule aquarium-yes ""
    ?f <- (aquarium yes) (not (rec ?)) =>
    (retract ?f)
    (assert (community
        (yesno "Do you want a community aquarium with several fish? (yes/no): "))))

(defrule aquarium-no ""
    ?f <- (aquarium no) (not (rec ?)) =>
    (retract ?f)
    (assert (slowexotic
        (yesno "Would you like a slow-moving, quiet exotic pet? (yes/no): "))))

```

```
(defrule playpet-yes ""
    ?f <- (playpet yes) (not (rec ?)) =>
    (retract ?f)
    (assert (talking
        (yesno "Do you want a bird that can talk or imitate sounds? (yes/no): "))))

(defrule playpet-no ""
    ?f <- (playpet no) (not (rec ?)) =>
    (retract ?f)
    (assert (bright
        (yesno "Do you like a bright or exotic-looking pet? (yes/no): "))))


; ============================================================
; 3.6  Recommendation rules  (one rule per leaf node)
;      Each also retracts its fact so working memory is
;      completely empty when the recommendation is made.
; ============================================================

(defrule trainable-yes   "" ?f <- (trainable yes)   (not (rec ?)) => (retract ?f) (assert (
    rec "Dog")))
(defrule trainable-no    "" ?f <- (trainable no)     (not (rec ?)) => (retract ?f) (assert (
    rec "Rabbit")))

(defrule independent-yes "" ?f <- (independent yes)  (not (rec ?)) => (retract ?f) (assert (
    rec "Cat")))
(defrule independent-no  "" ?f <- (independent no)   (not (rec ?)) => (retract ?f) (assert (
    rec "Guinea Pig")))

(defrule nocturnal-yes   "" ?f <- (nocturnal yes)    (not (rec ?)) => (retract ?f) (assert (
    rec "Hamster")))
(defrule nocturnal-no    "" ?f <- (nocturnal no)     (not (rec ?)) => (retract ?f) (assert (
    rec "Fancy Rat")))

(defrule quiet-yes       "" ?f <- (quiet yes)        (not (rec ?)) => (retract ?f) (assert (
    rec "Pigeon")))
(defrule quiet-no        "" ?f <- (quiet no)         (not (rec ?)) => (retract ?f) (assert (
    rec "Myna Bird")))

(defrule community-yes   "" ?f <- (community yes)    (not (rec ?)) => (retract ?f) (assert (
    rec "Aquarium Fish")))
(defrule community-no    "" ?f <- (community no)     (not (rec ?)) => (retract ?f) (assert (
    rec "Goldfish")))

(defrule slowexotic-yes  "" ?f <- (slowexotic yes)   (not (rec ?)) => (retract ?f) (assert (
    rec "Turtle")))
(defrule slowexotic-no   "" ?f <- (slowexotic no)    (not (rec ?)) => (retract ?f) (assert (
    rec "Cockatiel")))

(defrule talking-yes     "" ?f <- (talking yes)      (not (rec ?)) => (retract ?f) (assert (
    rec "Parrot")))
(defrule talking-no      "" ?f <- (talking no)       (not (rec ?)) => (retract ?f) (assert (
    rec "Canary")))

(defrule bright-yes      "" ?f <- (bright yes)       (not (rec ?)) => (retract ?f) (assert (
    rec "Lovebird")))
(defrule bright-no       "" ?f <- (bright no)        (not (rec ?)) => (retract ?f) (assert (
    rec "Budgerigar")))
```